src/components/CalculatorDemo.tsx 1/6


```
import React, { useState } from 'react'

interface CalculatorDemoProps {
  isOpen: boolean
  onClose: () => void
}

const CalculatorDemo: React.FC<CalculatorDemoProps> = ({ isOpen, onClose }) => {
  const [display, setDisplay] = useState('0')
  const [result, setResult] = useState(0)
  const [operator, setOperator] = useState<string | null>(null)
  const [input, setInput] = useState('')

  const handleInput = (val: string) => {
    if (val === '=') {
      if (operator && input) {
        const calcResult = evaluate(result, operator, parseFloat(input))
        setDisplay(calcResult.toString())
        setResult(calcResult)
        setInput('')
        setOperator(null)
      }
    } else if (['+', '-', '*', '/'].includes(val)) {
      if (input) {
        setResult(parseFloat(input))
      } else {
        setResult(result)
      }
      setOperator(val)
      setInput('')
    } else {
      setInput(input + val)
      setDisplay(input + val)
    }
  }

  const evaluate = (left: number, op: string, right: number): number => {
    switch (op) {
      case '+':
        return left + right
      case '-':
        return left - right
      case '*':
        return left * right
      case '/':
        return right !== 0 ? left / right : 0
      default:
        return right
    }
  }

  const handleClear = () => {
    setDisplay('0')
    setResult(0)
    setOperator(null)
    setInput('')
  }

  const buttons = [7, 8, 9, '/', 4, 5, 6, '*', 1, 2, 3, '-', 0, '.', '=', '+']

  return (
    <div className={`demo-modal ${isOpen ? 'open' : ''}`} onClick={(e) => e.target === e.currentTarget && onClose()}>
      <div className="demo-modal-content">
        <button className="demo-close" onClick={onClose}>
          &times;
        </button>

        <div className="demo-container">
          <header className="demo-header">
            <h1>ðŸ§® Calculatrice Scientifique</h1>
            <p>OpÃ©rations avancÃ©es intÃ©grÃ©es</p>
          </header>

          <main className="demo-main" style={{ textAlign: 'center', padding: '20px' }}>
            <div style={{ background: 'rgba(255,255,255,0.05)', border: '1px solid rgba(255,0,110,0.2)', borderRadius: '15px', padding: '15px' }}>
              <input
                type="text"
                readOnly
                value={display}
                style={{
                  width: '100%',
                  padding: '12px',
                  background: 'rgba(0,0,0,0.3)',
                  border: '1px solid rgba(255,0,110,0.3)',
                  borderRadius: '8px',
```

```
                color: '#ff006e',
                fontSize: '1.5rem',
                fontWeight: 'bold',
                marginBottom: '12px',
                textAlign: 'right',
              }}
            />
            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(4, 1fr)', gap: '8px', marginBottom: '12px' }}>
              {buttons.map((btn) => (
                <button
                  key={btn}
                  onClick={() => handleInput(btn.toString())}
                  style={{
                    padding: '14px',
                    background: ['/', '+', '-', '*', '='].includes(btn.toString())
                      ? 'rgba(255,0,110,0.2)'
                      : 'rgba(99,102,241,0.2)',
                    border: '1px solid rgba(255,0,110,0.3)',
                    borderRadius: '8px',
                    color: 'white',
                    fontWeight: 'bold',
                    fontSize: '1.1rem',
                    cursor: 'pointer',
                    transition: 'all 0.2s ease',
                  }}
                  onMouseEnter={(e) => {
                    (e.target as HTMLElement).style.background = 'rgba(255,0,110,0.3)'
                    ;(e.target as HTMLElement).style.transform = 'scale(1.05)'
                  }}
                  onMouseLeave={(e) => {
                    (e.target as HTMLElement).style.background = ['/', '+', '-', '*', '='].includes(btn.toString())
                      ? 'rgba(255,0,110,0.2)'
                      : 'rgba(99,102,241,0.2)'
                    ;(e.target as HTMLElement).style.transform = 'scale(1)'
                  }}
                >
                  {btn}
                </button>
              ))}
            </div>
            <button
              onClick={handleClear}
              style={{
                width: '100%',
                padding: '14px',
                background: 'rgba(239,68,68,0.2)',
                border: '1px solid rgba(239,68,68,0.3)',
                borderRadius: '8px',
                color: '#ef4444',
                fontWeight: 'bold',
                fontSize: '1.1rem',
                cursor: 'pointer',
              }}
            >
              C
            </button>
          </div>
        </main>
      </div>
    </div>
  )
}

export default CalculatorDemo
```

---

src/components/ContactForm.tsx 2/6

[top][prev][next]

```
import React, { useState } from 'react'

const ContactForm: React.FC = () => {
  const [name, setName] = useState('')
  const [email, setEmail] = useState('')
  const [message, setMessage] = useState('')
  const [loading, setLoading] = useState(false)
  const [status, setStatus] = useState<'idle' | 'success' | 'error'>('idle')
  const [statusMessage, setStatusMessage] = useState('')

  // Replace with your actual Formspree ID
  const FORMSPREE_ID = 'f/YOUR_FORMSPREE_ID' // You need to update this

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setLoading(true)
    setStatus('idle')
```

```
  try {
    const response = await fetch(`https://formspree.io/${FORMSPREE_ID}`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        name,
        email,
        message,
      }),
    })

    if (response.ok) {
      setStatus('success')
      setStatusMessage('â… Message envoyÃ© avec succÃ¨s! Merci de nous avoir contactÃ©.')
      setName('')
      setEmail('')
      setMessage('')
    } else {
      setStatus('error')
      setStatusMessage('â Erreur lors de l\'envoi du message. Veuillez rÃ©essayer.')
    }
  } catch (error) {
    setStatus('error')
    setStatusMessage('â Erreur de connexion. Veuillez vÃ©rifier votre connexion Internet.')
  } finally {
    setLoading(false)
  }
}

return (
  <section id="contact" className="section contact">
    <h2>Discutons de votre projet</h2>

    <div className="contact-container">
      {/* Bloc Informations (Gauche) */}
      <div className="contact-info">
        <h3>Contactez-moi</h3>
        <p>
          Vous avez une idÃ©e en tÃªte ou besoin d&apos;un dÃ©veloppeur pour votre Ã©quipe ? N&apos;hÃ©sitez pas Ã
          m&apos;envoyer un message.
        </p>

        <div className="info-item">
          <span className="icon">ðŸ“§</span>
          <div className="info-text">
            <h4>Email</h4>
            <p>giovannihunlede@gmail.com</p>
          </div>
        </div>

        <div className="info-item">
          <span className="icon">ðŸ“</span>
          <div className="info-text">
            <h4>Localisation</h4>
            <p>LomÃ© TOGO</p>
          </div>
        </div>
      </div>

      {/* Formulaire (Droite) */}
      <form className="contact-form" onSubmit={handleSubmit}>
        {status !== 'idle' && (
          <div className={`form-status form-status-${status}`}>
            {statusMessage}
          </div>
        )}

        <div className="input-group">
          <input
            type="text"
            id="name"
            value={name}
            onChange={(e) => setName(e.target.value)}
            required
            placeholder=" "
          />
          <label htmlFor="name">Votre Nom</label>
        </div>

        <div className="input-group">
          <input
            type="email"
            id="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            placeholder=" "
          />
          <label htmlFor="email">Votre Email</label>
```

```
          </div>

          <div className="input-group">
            <textarea
              id="message"
              value={message}
              onChange={(e) => setMessage(e.target.value)}
              rows={5}
              required
              placeholder=" "
            ></textarea>
            <label htmlFor="message">Votre Message</label>
          </div>

          <button type="submit" className="btn-submit" disabled={loading}>
            {loading ? 'Envoi en cours...' : 'Envoyer le message'} <span>ðŸš€</span>
          </button>
        </form>
      </div>
    </section>
  )
}

export default ContactForm
```

---

src/components/FlowNoteDemo.tsx 3/6

[top][prev][next]

```
import React, { useState, useEffect } from 'react'

interface Note {
  id: number
  title: string
  content: string
  category: string
  date: string
  time: string
}

interface FlowNoteDemoProps {
  isOpen: boolean
  onClose: () => void
}

const FlowNoteDemo: React.FC<FlowNoteDemoProps> = ({ isOpen, onClose }) => {
  const [notes, setNotes] = useState<Note[]>([])
  const [title, setTitle] = useState('')
  const [content, setContent] = useState('')
  const [category, setCategory] = useState('general')
  const [searchQuery, setSearchQuery] = useState('')
  const [selectedCategory, setSelectedCategory] = useState('all')

  // Load notes from localStorage
  useEffect(() => {
    const stored = localStorage.getItem('demoFlowNotes')
    if (stored) {
      setNotes(JSON.parse(stored))
    }
  }, [])

  // Save notes to localStorage
  const saveNotes = (newNotes: Note[]) => {
    setNotes(newNotes)
    localStorage.setItem('demoFlowNotes', JSON.stringify(newNotes))
  }

  const addNote = () => {
    if (!title.trim() || !content.trim()) {
      alert('Veuillez remplir le titre et le contenu!')
      return
    }

    const newNote: Note = {
      id: Date.now(),
      title,
      content,
      category,
      date: new Date().toLocaleDateString('fr-FR'),
      time: new Date().toLocaleTimeString('fr-FR', { hour: '2-digit', minute: '2-digit' }),
    }

    saveNotes([newNote, ...notes])
    setTitle('')
    setContent('')
  }

  const deleteNote = (id: number) => {
```

```
  if (confirm('Ã‰tes-vous sÃ»r?')) {
    saveNotes(notes.filter((note) => note.id !== id))
  }
}

const editNote = (id: number) => {
  const note = notes.find((n) => n.id === id)
  if (note) {
    setTitle(note.title)
    setContent(note.content)
    setCategory(note.category)
    deleteNote(id)
  }
}

const getCategoryColor = (cat: string) => {
  const colors: { [key: string]: string } = {
    work: '#FF6B6B',
    personal: '#4ECDC4',
    ideas: '#FFE66D',
    important: '#95E1D3',
    general: '#99D8FF',
  }
  return colors[cat] || '#99D8FF'
}

const escapeHtml = (text: string) => {
  const div = document.createElement('div')
  div.textContent = text
  return div.innerHTML
}

const filteredNotes = notes.filter((note) => {
  const matchesSearch =
    note.title.toLowerCase().includes(searchQuery.toLowerCase()) ||
    note.content.toLowerCase().includes(searchQuery.toLowerCase())
  const matchesCategory = selectedCategory === 'all' || note.category === selectedCategory
  return matchesSearch && matchesCategory
})

return (
  <div className={`demo-modal ${isOpen ? 'open' : ''}`} onClick={(e) => e.target === e.currentTarget && onClose()}>
    <div className="demo-modal-content">
      <button className="demo-close" onClick={onClose}>
        &times;
      </button>

      <div className="demo-container">
        <header className="demo-header">
          <h1>ðŸ“ FlowNote - DÃ©mo Interactive</h1>
          <p>Testez l&apos;app directement! Vos notes sont sauvegardÃ©es localement.</p>
        </header>

        <main className="demo-main">
          <div className="demo-input-section">
            <div className="demo-input-group">
              <input
                type="text"
                value={title}
                onChange={(e) => setTitle(e.target.value)}
                onKeyPress={(e) => e.key === 'Enter' && addNote()}
                placeholder="Titre de votre note..."
              />
              <textarea
                value={content}
                onChange={(e) => setContent(e.target.value)}
                onKeyPress={(e) => e.ctrlKey && e.key === 'Enter' && addNote()}
                placeholder="Contenu de votre note..."
              ></textarea>
              <div className="demo-form-row">
                <select value={category} onChange={(e) => setCategory(e.target.value)} className="demo-category-select">
                  <option value="general">ðŸ“Œ General</option>
                  <option value="work">ðŸ’¼ Travail</option>
                  <option value="personal">ðŸ‘¤ Personnel</option>
                  <option value="ideas">ðŸ’¡ IdÃ©es</option>
                  <option value="important">â Important</option>
                </select>
                <button className="demo-btn-add" onClick={addNote}>
                  âž• Ajouter
                </button>
              </div>
            </div>
          </div>

          <div className="demo-search-section">
            <input
              type="text"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              placeholder="ðŸ” Rechercher..."
              className="demo-search-input"
```

```
              />
              <select
                value={selectedCategory}
                onChange={(e) => setSelectedCategory(e.target.value)}
                className="demo-category-select"
              >
                <option value="all">Toutes catÃ©gories</option>
                <option value="general">ðŸ“Œ General</option>
                <option value="work">ðŸ’¼ Travail</option>
                <option value="personal">ðŸ‘¤ Personnel</option>
                <option value="ideas">ðŸ’¡ IdÃ©es</option>
                <option value="important">â Important</option>
              </select>
              <div className="demo-note-stats">
                <span>ðŸ“ Total: {notes.length}</span>
                <span>ðŸ” AffichÃ©es: {filteredNotes.length}</span>
              </div>
            </div>

            <div className="demo-notes-section">
              <h2>Vos Notes</h2>
              <div className="demo-notes-grid">
                {filteredNotes.length === 0 ? (
                  <div className="demo-empty-state">
                    <h3>{searchQuery ? 'âŒ Aucune note trouvÃ©e' : 'ðŸ“ Aucune note'}</h3>
                  </div>
                ) : (
                  filteredNotes.map((note) => (
                    <div
                      key={note.id}
                      className="demo-note-card"
                      style={{ borderLeftColor: getCategoryColor(note.category) }}
                    >
                      <div className="demo-note-header">
                        <h3>{escapeHtml(note.title)}</h3>
                        <span className="demo-note-category" style={{ backgroundColor: getCategoryColor(note.category) }}>
                          {note.category}
                        </span>
                      </div>
                      <p>{escapeHtml(note.content)}</p>
                      <div className="demo-note-meta">
                        <small>
                          ðŸ“… {note.date} Â· ðŸ• {note.time}
                        </small>
                      </div>
                      <div className="demo-note-actions">
                        <button className="demo-btn-edit" onClick={() => editNote(note.id)}>
                          âœï¸
                        </button>
                        <button className="demo-btn-delete" onClick={() => deleteNote(note.id)}>
                          ðŸ—‘ï¸
                        </button>
                      </div>
                    </div>
                  ))
                )}
              </div>
            </div>
          </main>
        </div>
      </div>
    </div>
  )
}

export default FlowNoteDemo
```

---

src/components/Modal.tsx 4/6

[top][prev][next]

```
import React, { useEffect } from 'react'

interface ModalProps {
  id: string
  isOpen: boolean
  onClose: () => void
  title: string
  children: React.ReactNode
}

const Modal: React.FC<ModalProps> = ({ id, isOpen, onClose, title, children }) => {
  useEffect(() => {
    const handleKeyDown = (e: KeyboardEvent) => {
      if (e.key === 'Escape' && isOpen) {
        onClose()
      }
    }
```

```
    if (isOpen) {
      document.body.style.overflow = 'hidden'
      document.addEventListener('keydown', handleKeyDown)
    }

    return () => {
      document.body.style.overflow = ''
      document.removeEventListener('keydown', handleKeyDown)
    }
  }, [isOpen, onClose])

  const handleBackdropClick = (e: React.MouseEvent) => {
    if (e.target === e.currentTarget) {
      onClose()
    }
  }

  return (
    <div
      id={id}
      className={`demo-modal ${isOpen ? 'open' : ''}`}
      onClick={handleBackdropClick}
    >
      <div className="demo-modal-content">
        <button className="demo-close" onClick={onClose}>
          &times;
        </button>

        <div className="demo-container">
          <header className="demo-header">
            <h1>{title}</h1>
          </header>

          <main className="demo-main">{children}</main>
        </div>
      </div>
    </div>
  )
}

export default Modal
```

---

src/components/SnakeGameDemo.tsx 5/6

[top][prev][next]

```
import React, { useState, useEffect, useRef } from 'react'

interface SnakeGameDemoProps {
  isOpen: boolean
  onClose: () => void
}

const SnakeGameDemo: React.FC<SnakeGameDemoProps> = ({ isOpen, onClose }) => {
  const canvasRef = useRef<HTMLCanvasElement>(null)
  const [score, setScore] = useState(0)
  const [gameOver, setGameOver] = useState(false)
  const [gameStarted, setGameStarted] = useState(false)

  const gameStateRef = useRef({
    snake: [{ x: 6, y: 6 }],
    direction: { x: 1, y: 0 },
    nextDir: { x: 1, y: 0 },
    food: { x: Math.floor(Math.random() * 12), y: Math.floor(Math.random() * 12) },
    score: 0,
    gameOver: false,
    width: 12,
    height: 12,
    cellSize: 30,
  })

  useEffect(() => {
    if (!isOpen || !gameStarted) return

    const handleKeyDown = (e: KeyboardEvent) => {
      const state = gameStateRef.current
      const key = e.key

      if (key === 'ArrowUp' && state.direction.y === 0) {
        state.nextDir = { x: 0, y: -1 }
        e.preventDefault()
      } else if (key === 'ArrowDown' && state.direction.y === 0) {
        state.nextDir = { x: 0, y: 1 }
        e.preventDefault()
      } else if (key === 'ArrowLeft' && state.direction.x === 0) {
        state.nextDir = { x: -1, y: 0 }
        e.preventDefault()
```

```
    } else if (key === 'ArrowRight' && state.direction.x === 0) {
      state.nextDir = { x: 1, y: 0 }
      e.preventDefault()
    }
  }

  document.addEventListener('keydown', handleKeyDown)
  return () => document.removeEventListener('keydown', handleKeyDown)
}, [isOpen, gameStarted])

useEffect(() => {
  if (!isOpen || !gameStarted || gameOver) return

  const canvas = canvasRef.current
  if (!canvas) return

  const ctx = canvas.getContext('2d')
  if (!ctx) return

  const state = gameStateRef.current

  const gameLoop = setInterval(() => {
    // Update
    state.direction = state.nextDir
    const head = {
      x: state.snake[0].x + state.direction.x,
      y: state.snake[0].y + state.direction.y,
    }

    // Check collisions
    if (head.x < 0 || head.x >= state.width || head.y < 0 || head.y >= state.height) {
      setGameOver(true)
      clearInterval(gameLoop)
      return
    }

    if (state.snake.some((s) => s.x === head.x && s.y === head.y)) {
      setGameOver(true)
      clearInterval(gameLoop)
      return
    }

    state.snake.unshift(head)

    if (head.x === state.food.x && head.y === state.food.y) {
      state.score += 10
      setScore(state.score)
      state.food = {
        x: Math.floor(Math.random() * state.width),
        y: Math.floor(Math.random() * state.height),
      }
    } else {
      state.snake.pop()
    }

    // Draw
    ctx.fillStyle = '#0a0e27'
    ctx.fillRect(0, 0, canvas.width, canvas.height)

    ctx.fillStyle = '#00d9ff'
    state.snake.forEach((s) => {
      ctx.fillRect(s.x * state.cellSize, s.y * state.cellSize, state.cellSize - 1, state.cellSize - 1)
    })

    ctx.fillStyle = '#ff006e'
    ctx.fillRect(state.food.x * state.cellSize, state.food.y * state.cellSize, state.cellSize - 1, state.cellSize - 1)
  }, 150)

  return () => clearInterval(gameLoop)
}, [isOpen, gameStarted, gameOver])

const startGame = () => {
  setGameStarted(true)
  setGameOver(false)
  setScore(0)
  gameStateRef.current = {
    snake: [{ x: 6, y: 6 }],
    direction: { x: 1, y: 0 },
    nextDir: { x: 1, y: 0 },
    food: { x: Math.floor(Math.random() * 12), y: Math.floor(Math.random() * 12) },
    score: 0,
    gameOver: false,
    width: 12,
    height: 12,
    cellSize: 30,
  }
}

return (
  <div className={`demo-modal ${isOpen ? 'open' : ''}`} onClick={(e) => e.target === e.currentTarget && onClose()}>
    <div className="demo-modal-content">
```

```
        <button className="demo-close" onClick={onClose}>
          &times;
        </button>

        <div className="demo-container">
          <header className="demo-header">
            <h1>ðŸ Snake Game - DÃ©mo Interactive</h1>
            <p>Utilisez les flÃ¨ches pour contrÃ´ler le serpent!</p>
          </header>

          <main className="demo-main" style={{ textAlign: 'center', padding: '20px' }}>
            {!gameStarted ? (
              <button
                onClick={startGame}
                style={{
                  padding: '14px 28px',
                  background: 'linear-gradient(135deg, #ff006e 0%, #00d9ff 100%)',
                  color: 'white',
                  border: 'none',
                  borderRadius: '12px',
                  fontWeight: '700',
                  fontSize: '1.1rem',
                  cursor: 'pointer',
                  transition: 'all 0.3s ease',
                  marginBottom: '20px',
                }}
                onMouseEnter={(e) => ((e.target as HTMLElement).style.transform = 'translateY(-2px)')}
                onMouseLeave={(e) => ((e.target as HTMLElement).style.transform = 'translateY(0)')}
              >
                ðŸŽ® DÃ©marrer le jeu
              </button>
            ) : (
              <>
                <canvas
                  ref={canvasRef}
                  width={12 * 30}
                  height={12 * 30}
                  style={{
                    border: '2px solid #ff006e',
                    backgroundColor: '#0a0e27',
                    marginBottom: '20px',
                    borderRadius: '12px',
                  }}
                />
                <div style={{ color: '#ff006e', fontSize: '1.2rem', fontWeight: 'bold', marginBottom: '20px' }}>
                  {gameOver ? `ðŸŽ® Game Over! Score: ${score}` : `Score: ${score}`}
                </div>
                {gameOver && (
                  <button
                    onClick={startGame}
                    style={{
                      padding: '12px 24px',
                      background: 'linear-gradient(135deg, #ff006e 0%, #00d9ff 100%)',
                      color: 'white',
                      border: 'none',
                      borderRadius: '12px',
                      fontWeight: '700',
                      cursor: 'pointer',
                    }}
                  >
                    ðŸ”„ Rejouer
                  </button>
                )}
              </>
            )}
          </main>
        </div>
      </div>
    </div>
  )
}

export default SnakeGameDemo
```

---

src/components/TaskManagerDemo.tsx 6/6

[top][prev][next]

```
import React, { useState, useEffect } from 'react'

interface Task {
  id: number
  title: string
  desc: string
  priority: 'low' | 'medium' | 'high'
  completed: boolean
}
```

```
interface TaskManagerDemoProps {
  isOpen: boolean
  onClose: () => void
}

const TaskManagerDemo: React.FC<TaskManagerDemoProps> = ({ isOpen, onClose }) => {
  const [tasks, setTasks] = useState<Task[]>([])
  const [title, setTitle] = useState('')
  const [desc, setDesc] = useState('')
  const [priority, setPriority] = useState<'low' | 'medium' | 'high'>('medium')

  useEffect(() => {
    const stored = localStorage.getItem('demoTasks')
    if (stored) {
      setTasks(JSON.parse(stored))
    }
  }, [])

  const saveTasks = (newTasks: Task[]) => {
    setTasks(newTasks)
    localStorage.setItem('demoTasks', JSON.stringify(newTasks))
  }

  const addTask = () => {
    if (!title.trim()) {
      alert('Veuillez entrer un titre!')
      return
    }

    const newTask: Task = {
      id: Date.now(),
      title,
      desc,
      priority,
      completed: false,
    }

    saveTasks([newTask, ...tasks])
    setTitle('')
    setDesc('')
    setPriority('medium')
  }

  const toggleTask = (id: number) => {
    saveTasks(tasks.map((t) => (t.id === id ? { ...t, completed: !t.completed } : t)))
  }

  const deleteTask = (id: number) => {
    saveTasks(tasks.filter((t) => t.id !== id))
  }

  const colors = { low: '#10b981', medium: '#f59e0b', high: '#ef4444' }
  const icons = { low: 'ðŸŸ¢', medium: 'ðŸŸ¡', high: 'ðŸ”´' }

  return (
    <div className={`demo-modal ${isOpen ? 'open' : ''}`} onClick={(e) => e.target === e.currentTarget && onClose()}>
      <div className="demo-modal-content">
        <button className="demo-close" onClick={onClose}>
          &times;
        </button>

        <div className="demo-container">
          <header className="demo-header">
            <h1>ðŸ“Š TaskManager - DÃ©mo Interactive</h1>
            <p>GÃ©rez vos tÃ¢ches avec prioritÃ©s!</p>
          </header>

          <main className="demo-main">
            <div className="demo-input-section">
              <div className="demo-input-group">
                <input
                  type="text"
                  value={title}
                  onChange={(e) => setTitle(e.target.value)}
                  onKeyPress={(e) => e.key === 'Enter' && addTask()}
                  placeholder="Titre de la tÃ¢che..."
                />
                <textarea
                  value={desc}
                  onChange={(e) => setDesc(e.target.value)}
                  placeholder="Description..."
                ></textarea>
                <div className="demo-form-row">
                  <select value={priority} onChange={(e) => setPriority(e.target.value as any)} className="demo-category-select">
                    <option value="low">ðŸŸ¢ Basse</option>
                    <option value="medium">ðŸŸ¡ Normal</option>
                    <option value="high">ðŸ”´ Haute</option>
                  </select>
                  <button className="demo-btn-add" onClick={addTask}>
                    â• Ajouter
                  </button>
```

```
              </div>
            </div>
          </div>

          <div className="demo-notes-section">
            <h2>Vos TÃ¢ches ({tasks.length})</h2>
            <div className="demo-notes-grid">
              {tasks.length === 0 ? (
                <div className="demo-empty-state">
                  <h3>ðŸ“ Aucune tÃ¢che</h3>
                </div>
              ) : (
                tasks.map((task) => (
                  <div
                    key={task.id}
                    className="demo-note-card"
                    style={{ borderLeftColor: colors[task.priority], opacity: task.completed ? 0.5 : 1 }}
                  >
                    <div className="demo-note-header">
                      <h3 style={{ textDecoration: task.completed ? 'line-through' : 'none' }}>{task.title}</h3>
                      <span className="demo-note-category" style={{ backgroundColor: colors[task.priority] }}>
                        {icons[task.priority]} {task.priority}
                      </span>
                    </div>
                    <p>{task.desc}</p>
                    <div className="demo-note-actions">
                      <button className="demo-btn-edit" onClick={() => toggleTask(task.id)} style={{ flex: 1 }}>
                        {task.completed ? 'â®ï¸' : 'âœ…'}
                      </button>
                      <button className="demo-btn-delete" onClick={() => deleteTask(task.id)} style={{ flex: 1 }}>
                        ðŸ—‘ï¸
                      </button>
                    </div>
                  </div>
                ))
              )}
            </div>
          </div>
        </main>
      </div>
    </div>
  </div>
  )
}

export default TaskManagerDemo
```

Generated by [GNU Enscript 1.6.5.90](#).